Subject: OPTIMIZATION TECHNIQUES

Subject Code: 23AS2402e

UNIT I : Introduction and Linear Programming

Syllabus: Meaning, nature, scope and significance of optimization, typical applications; formulation of LPP, limitations, graphical method, simplex method (maximization and minimization), Big-M method and Two-Phase method.

I. Introduction to Optimization

Definition:

Optimization is the process of finding the best possible solution (maximum or minimum) of an objective, from all feasible alternatives, subject to given restrictions (constraints).

- Objective: maximise profit, revenue, efficiency; minimise cost, time, waste, risk.
- Origin: Operations Research (O.R.) developed in World War II (UK, 1940) for scarce military resources.
- O.R. = scientific, quantitative approach to decision making; uses mathematical models.

Nature of optimization

- Has a decision (controllable) variable set, an objective function, and constraints.
- Mathematical model: Optimise $Z = f(x1, \ldots, xn)$ subject to $g_i(x) \leq , = , \geq b_i$.
- Deals with limited resources: men, machines, materials, money, time.
- Interdisciplinary: mathematics, economics, engineering, statistics, computers.

Scope and significance

- Scope: production planning, inventory, finance and investment, marketing, transport and logistics, personnel, project planning, agriculture, defence, health care.
- Gives scientific basis for decisions instead of guess work; saves cost and time.
- Helps in resource allocation, co-ordination of departments and long term planning.

Typical applications

Area	Application
Production	Product mix, blending, job sequencing
Transport	Minimum cost shipping, vehicle routes
Finance	Portfolio selection, capital budgeting
Marketing	Media selection, advertising budget
Personnel	Assignment of staff, shift scheduling
Projects	CPM / PERT scheduling

Phases of an O.R. study

- 1) Define the problem 2) Construct the model 3) Solve the model 4) Validate the model 5) Implement the solution.

2. Linear Programming Problem (LPP) – Formulation

Definition:

Linear Programming is a technique for optimising a ***linear*** objective function subject to ***linear*** equality / inequality constraints, with non-negative variables.

General mathematical form

Formula:

Max (or Min) $Z = c_1x_1 + c_2x_2 + \dots + c_nx_n$ (objective function)

subject to $a_{i1}x_1 + a_{i2}x_2 + \dots + a_{in}x_n \ (\leq, =, \geq)\ b_i$, $i = 1, \dots, m$ (constraints)

$x_1, x_2, \dots, x_n \geq 0$ (non-negativity restriction)

- Decision variables x_j; cost / profit coefficients c_j; technological coefficients a_{ij}; right hand side (resources) b_i.
- Feasible solution: satisfies all constraints. Feasible region: set of all feasible solutions.
- Optimal solution: feasible solution that gives the best value of Z. Basic feasible solution: at most m non-zero variables.

Assumptions of LPP

- Linearity (proportionality and additivity), certainty (constants known), divisibility (fractions allowed), non-negativity, single objective.

Steps in formulation

- 1) Identify the decision variables. 2) Write the objective function. 3) Write constraints from resources. 4) Add $x \geq 0$.

Limitations of LPP

- Needs linear relations – real life is often non-linear.
- Assumes certainty of coefficients; no uncertainty / probability.
- Answers may be fractional (integer programming needed for whole numbers).
- Only one objective (goal programming for many); large problems need computers.

Example:

Product mix: A firm makes tables (profit Rs 40) and chairs (Rs 30). Each table needs 2 h of carpentry and each chair 1 h; 16 h are available. Total production capacity is 12 units. Formulate.

$x1$ = tables, $x2$ = chairs. Max $Z = 40x1 + 30x2$

s.t. $2x1 + x2 \leq 16$ (carpentry), $x1 + x2 \leq 12$ (capacity), $x1, x2 \geq 0$

Example:

Diet problem: A diet needs at least 18 units of vitamin V1 and 12 units of V2. Food 1 gives (6, 1) units per kg at Rs 20/kg; food 2 gives (1, 4) units per kg at Rs 40/kg. Formulate.

Min $Z = 20x1 + 40x2$ s.t. $6x1 + x2 \geq 18$, $x1 + 4x2 \geq 12$, $x1, x2 \geq 0$

Example:

Blending: Mix two ores to make at least 30 kg of alloy with at most 40 % impurity. Let $x1$, $x2$ be kg used: $x1 + x2 \geq 30$; $0.5x1 + 0.3x2 \leq 0.4(x1 + x2)$ $\Rightarrow 0.1x1 - 0.1x2 \leq 0$.

Note:

Check units of every constraint and make sure each variable appears in the objective.

3. Graphical Method (two variables)

- Step 1: Convert each inequality to an equation and draw the line.
- Step 2: Mark the side that satisfies the inequality; common area = feasible region.
- Step 3: Find the corner (extreme) points of the feasible region.

- Step 4: Evaluate Z at every corner point. Step 5: Max → largest Z, Min → smallest Z.

Note:

Fundamental theorem: if an optimal solution exists, it occurs at a corner point of the (convex) feasible region.

Example:

Solve: Max Z = 40x1 + 30x2 s.t. x1 + x2 ≤ 12, 2x1 + x2 ≤ 16, x1, x2 ≥ 0.

- Lines: x1 + x2 = 12 meets axes at (12, 0), (0, 12); 2x1 + x2 = 16 meets axes at (8, 0), (0, 16).
- Intersection: subtract the equations ⇒ x1 = 4, x2 = 8.

Corner point	Z = 40x1 + 30x2
O (0, 0)	0
A (8, 0)	320
B (4, 8)	400
C (0, 12)	360

- ***Optimal: x1 = 4, x2 = 8, Max Z = 400.***

Example:

Solve (diet): Min Z = 20x1 + 40x2 s.t. 6x1 + x2 ≥ 18, x1 + 4x2 ≥ 12, 2x1 + x2 ≥ 10.

- Feasible region is unbounded, lying above all the lines. Corner points: (0, 18), (2, 6), (4, 2), (12, 0).
- Intersections: 6x1 + x2 = 18 and 2x1 + x2 = 10 ⇒ (2, 6); 2x1 + x2 = 10 and x1 + 4x2 = 12 ⇒ (4, 2).

Corner point	Z = 20x1 + 40x2
(0, 18)	720

(2, 6)	280
(4, 2)	160
(12, 0)	240

- **Optimal: x1 = 4, x2 = 2, Min Z = 160.**

Special cases

- **Multiple optimal solutions:** objective line parallel to a binding constraint – every point on that edge is optimal (e.g. Max Z = 4x1 + 2x2 with 2x1 + x2 ≤ 8).
- **Unbounded solution:** feasible region unbounded in the direction of improvement; Z can be increased without limit.
- **Infeasible solution:** no common region (constraints contradict).
- **Redundant constraint:** does not touch the feasible region – can be dropped.

4. Simplex Method – Preliminaries

Definition:

Simplex method (G. B. Dantzig, 1947) is an iterative algebraic procedure that moves from one basic feasible solution (corner) to a better one until optimum is reached.

Standard form – convert all constraints to equations

Constraint	Add	Cost in objective
≤	slack variable s (+s)	0
≥	surplus variable (−s) and artificial A	0 and M (Big-M)
=	artificial variable A (+A)	M

- All b_i must be ≥ 0 (multiply by -1 if necessary, reversing the inequality). Unrestricted variable $x = x' - x''$.

Terms

- ***Basic variables:*** variables in the basis (value from RHS); non-basic variables are zero.
- C_B: cost of basic variables; $Z_j = \Sigma C_B \cdot$ (column j); net evaluation $\Delta_j = C_j - Z_j$.
- ***Key (pivot) column:*** entering variable. ***Key row:*** leaving variable (min ratio). ***Pivot element:*** intersection.

Simplex algorithm

- Step 1: Write the standard form and the initial table (slack variables form the initial basis).
- Step 2: Compute Δ_j. ***Max problem:*** optimal if all $\Delta_j \leq 0$. ***Min problem:*** optimal if all $\Delta_j \geq 0$.
- Step 3: Entering variable = most positive Δ_j (max) / most negative (min).
- Step 4: Ratio = RHS / (positive entries of key column); smallest ratio gives the leaving variable.
- Step 5: New key row = old key row ÷ pivot. Other rows: new row = old row − (its key column entry) × new key row.
- Step 6: Repeat until optimal.

5. Simplex Method – Maximisation Example

Example:

Max $Z = 3x1 + 5x2$ s.t. $x1 \leq 4$, $2x2 \leq 12$, $3x1 + 2x2 \leq 18$, $x1, x2 \geq 0$.

- Standard form: $x1 + s1 = 4$; $2x2 + s2 = 12$; $3x1 + 2x2 + s3 = 18$. Cj = 3, 5, 0, 0, 0.

- **Table 1** (Δj = 3, 5, 0, 0, 0 → x2 enters; ratios 12/2 = 6, 18/2 = 9 → s2 leaves, pivot 2)

Basis	x1	x2	s1	s2	s3	RHS
s1	1	0	1	0	0	4
s2	0	**2**	0	1	0	12
s3	3	2	0	0	1	18
Δj	3	5	0	0	0	$\bar{z}$ = 0

- **Table 2** (Δj = 3, 0, 0, -2.5, 0 → x1 enters; ratios 4/1 = 4, 6/3 = 2 → s3 leaves, pivot 3)

Basis	x1	x2	s1	s2	s3	RHS
s1	1	0	1	0	0	4
x2	0	1	0	1/2	0	6
s3	**3**	0	0	-1	1	6
Δj	3	0	0	-5/2	0	$\bar{z}$ = 30

- **Table 3** (Δj = 0, 0, 0, -3/2, -1 → all ≤ 0 ⇒ **optimal**)

Basis	x1	x2	s1	s2	s3	RHS
s1	0	0	1	1/3	-1/3	2
x2	0	1	0	1/2	0	6
x1	1	0	0	-1/3	1/3	2
Δj	0	0	0	-3/2	-1	$\bar{z}$ = 36

- **Optimal solution: x1 = 2, x2 = 6, Max Z = 36.**

Note:
All $\Delta j \leq 0$ but one non-basic variable with $\Delta j = 0$ ⇒ alternative optimal solutions exist.

6. Simplex Method – Minimisation Model

- **Method 1:** convert to a maximisation problem: Min $Z = -\text{Max}(-Z)$. Solve Max $Z' = -Z$, then Min $Z = -Z'$.
- **Method 2:** use the same tables but enter the variable with the most negative Δj and stop when all $\Delta j \geq 0$ (Big-M if $\geq$ constraints).

Example:

Min $Z = -2x1 - x2$ s.t. $x1 + x2 \leq 5$, $x1 \leq 3$, $x1, x2 \geq 0$.

- Put $Z' = -Z = 2x1 + x2$ and maximise. Standard: $x1 + x2 + s1 = 5$; $x1 + s2 = 3$.

Basis	x1	x2	s1	s2	RHS
s1	1	1	1	0	5
s2	1	0	0	1	3
Δj	2	1	0	0	$Z' = 0$

- x1 enters, ratios 5 and 3 ⇒ s2 leaves.

Basis	x1	x2	s1	s2	RHS
s1	0	1	1	-1	2
x1	1	0	0	1	3
Δj	0	1	0	-2	$Z' = 6$

- x2 enters; ratio 2 ⇒ s1 leaves.

Basis	x1	x2	s1	s2	RHS
x2	0	1	1	-1	2
x1	1	0	0	1	3
Δj	0	0	-1	-1	$Z' = 8$

- Optimal: $x1 = 3$, $x2 = 2$, $Z' = 8$ ⇒ **Min $Z = -8$.**

Special situations in simplex

- **Unbounded:** entering column has no positive entries (no ratio) ⇒ Z unbounded.
- **Degeneracy:** tie in minimum ratio ⇒ a basic variable becomes 0 (cycling is rare).
- **Infeasible:** optimal table still contains an artificial variable with positive value.
- **Alternative optima:** a non-basic variable has $\Delta j = 0$ at optimum.

7. Big-M Method (Charnes' penalty method)

Definition:

Used when the constraints include ≥ or = (no obvious starting basis). Artificial variables are added with a very large penalty M in the objective (+M for Min, -M for Max).

- Artificial variables are forced out of the basis by the penalty; once an artificial variable leaves, its column is dropped.
- If an artificial variable stays in the final basis at positive level ⇒ the problem has no feasible solution.

Example:

Min Z = 4x1 + x2 s.t. 3x1 + x2 = 3, 4x1 + 3x2 ≥ 6, x1 + 2x2 ≤ 4, x1, x2 ≥ 0.

- Standard form: 3x1 + x2 + A1 = 3; 4x1 + 3x2 − S2 + A2 = 6; x1 + 2x2 + S3 = 4. Cj = 4, 1, 0, 0, M, M.
- Initial basis A1, A2, S3. $\Delta x1 = 4 - 7M$, $\Delta x2 = 1 - 4M$, $\Delta S2 = M$. x1 enters (most negative); ratios 1, 1.5, 4 ⇒ A1 leaves.

Basis	xl	x2	S2	S3	A2	RHS
x1	1	1/3	0	0	0	1

A2	0	5/3	-1	0	1	2
S3	0	5/3	0	1	0	3

- $\Delta x2 = -(1 + 5M)/3$ is most negative: x2 enters; ratios 3, 6/5, 9/5 ⇒ A2 leaves (its column is dropped).

Basis	x1	x2	S2	S3	RHS
x1	1	0	1/5	0	3/5
x2	0	1	-3/5	0	6/5
S3	0	0	1	1	1

- $\Delta S2 = -1/5$ (negative) ⇒ S2 enters; ratios 3, -, 1 ⇒ S3 leaves.

Basis	x1	x2	S2	S3	RHS
x1	1	0	0	-1/5	2/5
x2	0	1	0	3/5	9/5
S2	0	0	1	1	1

- $\Delta S3 = 1/5 \geq 0$ ⇒ **optimal: x1 = 2/5, x2 = 9/5, Min Z = 17/5 = 3.4.**

Note:

Check: $3(0.4) + 1.8 = 3$ ✓; $4(0.4) + 3(1.8) = 7 \geq 6$ ✓; $0.4 + 3.6 = 4$ ✓.

8. Two-Phase Method

Definition:

Avoids the large number M. **Phase I** finds a basic feasible solution; **Phase II** optimises the original objective.

- **Phase I:** replace the objective by W = sum of artificial variables, Min W (cost 0 for all real variables, 1 for each artificial).
  - If Min W = 0 → feasible solution exists, go to Phase II. If Min W > 0 → problem infeasible, stop.
- **Phase II:** take the final Phase I table (drop artificial columns), restore the original Cj, recompute Δj and continue simplex.

Example:

Same problem as before: Min Z = 4x1 + x2 with the three constraints.

- **Phase I:** Min W = A1 + A2. Initial Δx1 = -7, Δx2 = -4, ΔS2 = +1 ⇒ x1 enters, A1 leaves.
- Then Δx2 = -5/3 ⇒ x2 enters, A2 leaves. Table: x1 = 3/5, x2 = 6/5, S3 = 1, both artificials out ⇒ **Min W = 0**.

Basis	x1	x2	S2	S3	RHS
x1	1	0	1/5	0	3/5
x2	0	1	-3/5	0	6/5
S3	0	0	1	1	1

- **Phase II:** Cj = 4, 1, 0, 0. ΔS2 = 0 – (4/5 – 3/5) = -1/5 ⇒ S2 enters, S3 leaves.
- Next table gives ΔS3 = 1/5 ≥ 0 ⇒ optimal: **x1 = 2/5, x2 = 9/5, Z = 17/5** (same answer as Big-M).

Big-M	Two-Phase
One big table, M appears in Δj	Two simple tables, numbers only
Computer round-off with M	No M needed – more stable
Same final answer	Same final answer

9. Practice Problems with Answers

- 1) Max $Z = 5x1 + 4x2$ s.t. $6x1 + 4x2 \leq 24$, $x1 + 2x2 \leq 6$, $x1, x2 \geq 0$ (graphical). Ans: corners (0,0), (4,0), (3, 1.5), (0,3); $Z = 0, 20, 21, 12 \Rightarrow x1 = 3, x2 = 1.5, Z = 21$.
- 2) Max $Z = 2x1 + 3x2$ s.t. $x1 + x2 \leq 4$, $x1 + 3x2 \leq 6$. Ans: corners (0,0), (4,0), (3, 1), (0,2): $Z = 0, 8, 9, 6 \Rightarrow Z = 9$ at (3, 1).
- 3) Min $Z = 2x1 + 3x2$ s.t. $x1 + x2 \geq 5$, $x1 \geq 2$. Ans: corners (2,3), (5,0): $Z = 13, 10 \Rightarrow$ Min $Z = 10$ at (5, 0).
- 4) Why is the simplex ratio test required? Ans: to keep every basic variable ≥ 0 (feasibility).
- 5) Formulate: A firm needs at least 10 units of A and 12 units of B. Machine 1 gives 2A + 3B per hour (cost Rs 5/h), Machine 2 gives 4A + 2B (Rs 4/h).
  - Ans: Min $Z = 5x1 + 4x2$ s.t. $2x1 + 4x2 \geq 10$, $3x1 + 2x2 \geq 12$, $x1, x2 \geq 0$.

> **Note:**
> Duality is excluded from this syllabus – concentrate on tabular method, Big-M and Two-Phase.

Quick Revision – Unit I

- LPP = linear Z, linear constraints, $x \geq 0$. Optimum lies at a corner point.
- $\leq \rightarrow$ slack (cost 0); $\geq \rightarrow$ surplus + artificial; $= \rightarrow$ artificial.
- Max optimal when all $\Delta j \leq 0$; Min optimal when all $\Delta j \geq 0$; $\Delta j = Cj - Zj$.
- Leaving variable = minimum positive ratio (RHS / key column).
- Big-M: artificial cost M. Two-Phase: Phase I Min W (Σ artificial), Phase II original Z.

Subject: OPTIMIZATION TECHNIQUES

Subject Code: 23AS2402e

UNIT II : Transportation and Assignment

Syllabus: Transportation model, initial basic feasible solutions, optimality (MODI), unbalanced problems, maximization, degeneracy; assignment problem, Hungarian algorithm, multiple solutions, unbalanced and maximization cases.

I. Transportation Problem – Introduction

Definition:

The transportation problem is a special LPP that finds the least-cost plan to ship a single commodity from m sources (origins, supply a_i) to n destinations (demand b_j).

- Cost c_{ij} = cost of shipping one unit from source i to destination j. x_{ij} = units shipped.

Mathematical model

Formula:

Min $Z = \Sigma \Sigma c_{ij} x_{ij}$ $(i = 1..m, j = 1..n)$

$\Sigma_j x_{ij} = a_i$ (supply), $\Sigma_i x_{ij} = b_j$ (demand), $x_{ij} \geq 0$

- **Balanced** problem: $\Sigma a_i = \Sigma b_j$. Unbalanced otherwise.
- A balanced problem always has a feasible solution; a basic feasible solution has exactly $m + n - 1$ allocations (occupied cells).

Transportation table (running example)

From / To	D1	D2	D3	D4	Supply
S1	19	30	50	10	7
S2	70	30	40	60	9

S3	40	8	70	20	18
Demand	5	8	7	14	34

Note:

Total supply = 7 + 9 + 18 = 34 = total demand 5 + 8 + 7 + 14 → balanced. m + n − 1 = 3 + 4 − 1 = 6 allocations are needed.

Steps of the solution

- Step 1: Check balance (add a dummy if not). Step 2: Find an initial basic feasible solution (IBFS).
- Step 3: Test optimality (MODI / u-v method). Step 4: Improve using a loop until all opportunity costs ≥ 0.

2. Initial Basic Feasible Solution (IBFS)

(a) North-West Corner Rule (NWCR)

- Start at the top-left cell; allocate min(supply, demand); cross out the exhausted row / column; move right or down.
- Fast but ignores costs – usually far from optimal.

Example:

Apply NWC to the running example.

From / To	D1	D2	D3	D4	Supply
S1	5	2	-	-	7
S2	-	6	3	-	9
S3	-	-	4	14	18
Demand	5	8	7	14	34

- Cost = 5×19 + 2×30 + 6×30 + 3×40 + 4×70 + 14×20 = 95 + 60 + 180 + 120 + 280 + 280 = **Rs 1015**.

(b) Least Cost Method (Matrix Minima)

- Allocate as much as possible in the cell with the smallest cost; cross out the satisfied row / column; repeat on the remaining cells.

> **Example:**
> Smallest cost is 8 (S3, D2): allocate 8. Next 10 (S1, D4): 7. Next 20 (S3, D4): 7. Then 40: (S2, D3): 7, (S3, D1): 3, (S2, D1): 2.

- Allocations: 8×8 + 7×10 + 7×20 + 7×40 + 3×40 + 2×70 = 64 + 70 + 140 + 280 + 120 + 140 = **Rs 814** (6 allocations ✓).

(c) Vogel's Approximation Method (VAM) – best IBFS

- Step 1: For every row and column find the **penalty** = difference between the two smallest costs.
- Step 2: Select the row / column with the largest penalty; allocate maximum in its lowest cost cell.
- Step 3: Cross out the satisfied row / column, recompute penalties, repeat. (Ties: pick the lowest cost cell.)

Iteration	Largest penalty	Allocation made
1	Column D2 (22)	S3–D2 = 8
2	Column D1 (21)	S1–D1 = 5
3	Row S3 (50)	S3–D4 = 10
4	Column D4 (50)	S1–D4 = 2
5	remaining	S2–D4 = 2, S2–D3 = 7

- Row penalties (initial): S1 = 9, S2 = 10, S3 = 12. Column penalties: D1 = 21, D2 = 22, D3 = 10, D4 = 10.
- Cost = 8×8 + 5×19 + 10×20 + 2×10 + 2×60 + 7×40 = 64 + 95 + 200 + 20 + 120 + 280 = **Rs 779**.

Method	Cost	Quality
North-West Corner	1015	worst (ignores cost)
Least Cost	814	better
VAM	779	closest to optimal

3. Moving Towards Optimality - MODI (u-v) Method

- Requires m + n - 1 occupied cells. For occupied cells: $u_i + v_j = c_{ij}$ (set u1 = 0 to start).
- For each empty cell compute the opportunity cost (net evaluation): $\Delta_{ij} = c_{ij} - (u_i + v_j)$.
- If all $\Delta_{ij} \geq 0$ → optimal. If some $\Delta_{ij} < 0$ → improve: choose the most negative cell.

Improvement by closed loop

- Draw a closed loop (horizontal / vertical moves) from the entering cell through occupied cells; mark +, -, +, - alternately.
- θ = smallest allocation on the minus cells; add θ to plus cells, subtract from minus cells. Re-test.

Example:

Test the VAM solution (cost 779) for optimality.

- Occupied: (S1,D1)=19, (S1,D4)=10, (S2,D3)=40, (S2,D4)=60, (S3,D2)=8, (S3,D4)=20.
- u1 = 0 ⇒ v1 = 19, v4 = 10; u2 = 60 - 10 = 50 ⇒ v3 = 40 - 50 = -10; u3 = 20 - 10 = 10 ⇒ v2 = 8 - 10 = -2.

Empty cell	c	u + v	Δ = c - (u + v)
S1-D2	30	-2	32
S1-D3	50	-10	60

S2-D1	₹0	69	1
S2-D2	30	48	**-18**
S3-D1	40	29	11
S3-D3	₹0	0	₹0

- Only S2-D2 is negative (-18) ⇒ not optimal. Loop: S2-D2 (+), S2-D4 (-), S3-D4 (+), S3-D2 (-).
- Minus cells hold 2 (S2-D4) and 8 (S3-D2) ⇒ θ = 2. S2-D2 = 2, S2-D4 = 0, S3-D4 = 12, S3-D2 = 6.

From / To	D1	D2	D3	D4	Supply
S1	5	-	-	2	₹
S2	-	2	₹	-	9
S3	-	6	-	12	18
Demand	5	8	₹	14	34

- New cost = 5×19 + 2×10 + 2×30 + ₹×40 + 6×8 + 12×20 = 95 + 20 + 60 + 280 + 48 + 240 = **743** (= ₹79 - 18 × 2 ✓).
- Re-test: u1 = 0, v1 = 19, v4 = 10, u3 = 10, v2 = -2, u2 = 32, v3 = 8. All Δ for empty cells: 32, 42, 19, 18, 11, 52 ≥ 0 ⇒ ***optimal, Min cost = Rs 743***.

> **Note:**
> Cost change after a loop = θ × Δ (here 2 × -18 = -36). Zero Δ for an empty cell ⇒ alternative optimal solution.

4. Unbalanced, Maximisation and Degeneracy

Unbalanced transportation problem

- If supply > demand: add a ***dummy destination*** with demand = excess supply, all costs 0.

- If demand > supply: add a ***dummy source*** with supply = shortage, all costs 0.
- Solve normally; shipments to a dummy mean unused capacity / unmet demand.

Example:

Supply 20, 30, 25 (total 75); demand 25, 30, 10 (total 65). Excess supply 10.

From / To	D1	D2	D3	Dummy	Supply
S1	4	6	8	0	20
S2	5	3	7	0	30
S3	6	4	5	0	25
Demand	25	30	10	10	75

- Now balanced (75 = 75); solve by VAM / MODI as usual.

Maximisation problem

- Objective is profit: convert to minimisation by subtracting every entry from the largest entry (opportunity loss matrix), or multiply the matrix by -1.
- Solve as a minimisation; the optimum profit = Σ (original profit $\times$ x_{ij}).

Example:

Profit matrix: largest element 20 → new cost = 20 - profit, then solve by VAM and MODI.

Degeneracy in transportation

Definition:

A basic feasible solution is degenerate if the number of occupied cells is less than m + n - 1.

- Occurs when a row and a column are satisfied at the same time during allocation.

- Remedy: allocate a very small quantity ε (ε → 0) to an empty cell such that no closed loop is formed (an independent position) and treat it as occupied.
- During optimality, ε may also appear when a minus cell ties for θ: keep one as ε.
- At the end put ε = 0 in the final answer.

Note:
Check the number of allocations = m + n – 1 before every MODI test.

5. Assignment Problem – Introduction

Definition:
The assignment problem assigns n jobs to n workers (machines) one-to-one so that total cost (time) is minimum or total profit is maximum.

- It is a special transportation problem with all supplies = demands = 1.

Mathematical formulation

Formula:
Min $Z = \sum \sum c_{ij} x_{ij}$, x_{ij} = 1 if worker i is assigned to job j, 0 otherwise
$\sum_j x_{ij} = 1$ (each worker one job), $\sum_i x_{ij} = 1$ (each job one worker)

- Solved by the ***Hungarian method*** (Kuhn, 1955; based on KÖnig's theorem).
- Key property: subtracting a constant from any row or column does not change the optimal assignment.

6. Hungarian Algorithm

- Step 1: Make the matrix square (add dummy row / column with zeros if unbalanced).

- Step 2: ***Row reduction*** – subtract the smallest element of each row from that row.
- Step 3: ***Column reduction*** – subtract the smallest element of each column from that column.
- Step 4: Cover all zeros with the ***minimum number of horizontal and vertical lines***.
- Step 5: If number of lines = n → optimal assignment possible (go to step 7).
- Step 6: If lines < n: find the smallest uncovered element k; subtract k from every uncovered element, add k to elements at line intersections; return to step 4.
- Step 7: Assign at zeros: start with rows / columns having a single zero, cross out the row and column, continue.

Example:

Assign 4 jobs to 4 men to minimise cost.

Man / Job	J1	J2	J3	J4
A	10	12	19	11
B	5	10	7	8
C	12	14	13	11
D	8	15	11	9

- Row minima 10, 5, 11, 8 ⇒

Man	J1	J2	J3	J4
A	0	2	9	1
B	0	5	2	3
C	1	3	2	0
D	0	7	3	1

- Column minima 0, 2, 2, 0 ⇒

Man	J1	J2	J3	J4
A	0	0	~~7~~	1
B	0	3	0	3
C	1	1	0	0
D	0	5	1	1

- Four lines needed (n = 4) ⇒ assignment: A → J2, B → J3, C → J4, D → J1 (unique zeros).
- Cost = 12 + ~~7~~ + 11 + 8 = **38**.

7. Hungarian Method – Case with Fewer Lines

Example:

Minimise the cost.

Man / Job	J1	J2	J3	J4
A	1	4	6	3
B	9	~~7~~	10	9
C	4	5	11	~~7~~
D	8	~~7~~	8	5

- Row minima 1, ~~7~~, 4, 5 ⇒ rows: A (0 3 5 2), B (2 0 3 2), C (0 1 ~~7~~ 3), D (3 2 3 0).
- Column minima 0, 0, 3, 0 ⇒ subtract 3 from column 3.

Man	J1	J2	J3	J4
A	0	3	2	2
B	2	0	0	2
C	0	1	4	3
D	3	2	0	0

- Zeros can be covered by 3 lines (column J1, row B, row D) < 4 ⇒ improve.
- Smallest uncovered element = 1 (cell C-J2). Subtract 1 from uncovered, add 1 at intersections (B-J1, D-J1).

Man	J1	J2	J3	J4
A	0	2	1	1
B	3	0	0	2
C	0	0	3	2
D	4	2	0	0

- Now 4 lines needed. A has a single zero ⇒ A → J1; C → J2; B → J3; D → J4.
- Cost = 1 + 5 + 10 + 5 = **21**.

> **Note:**
> If at the end a row (or column) has more than one zero in all choices, more than one optimal assignment exists (multiple solutions) – all have the same cost.

8. Multiple Solutions, Unbalanced and Maximisation

Multiple optimal solutions

- When, after reduction, some rows / columns have two zeros and the assignment can be made in different ways → alternative solutions with equal cost. Write each.

> **Example:**
> Reduced matrix rows (0 0 3), (0 2 0), (3 0 0) ⇒ A→1, B→3, C→2 or A→2, B→1, C→3: two optimal assignments.

Unbalanced assignment problem

- Number of workers ≠ number of jobs: add dummy jobs / workers with zero cost to get a square matrix.
- A worker assigned to a dummy job stays idle; a job assigned to a dummy worker is not done.

Example:
3 workers, 4 jobs: add a dummy worker row (0, 0, 0, 0) and apply Hungarian method to the 4 × 4 matrix.

Maximisation (profit) assignment

- Either multiply the matrix by −1 and add a constant to remove negatives, or subtract every element from the largest element (regret matrix); then minimise.

Example:
Profit matrix for 3 salesmen and 3 areas:

Salesman	Area 1	Area 2	Area 3
P	12	10	8
Q	9	11	7
R	10	6	13

- Largest = 13 ⇒ regret (13 − profit):

Salesman	Area 1	Area 2	Area 3
P	1	3	5
Q	4	2	6
R	3	7	0

- Row minima 1, 2, 0 ⇒ (0 2 4), (2 0 4), (3 7 0); column minima are 0 ⇒ assignment: P → 1, Q → 2, R → 3.
- Maximum profit = 12 + 11 + 13 = **36**.

Restricted assignment

- If a worker cannot do a job, put a very large cost M in that cell.

9. Practice Problems with Answers

- 1) Find NWC cost: supply 20, 30; demand 10, 25, 15; costs row1 (2, 3, 1), row2 (5, 4, 8). Ans: x11 = 10, x12 = 10, x22 = 15, x23 = 15; cost = 20 + 30 + 60 + 120 = 230.
- 2) How many occupied cells are required for a 4 × 5 transportation table? Ans: 4 + 5 – 1 = 8.
- 3) What is done if supply exceeds demand? Ans: add a dummy destination with zero costs.
- 4) Solve the assignment (min): rows (9, 11, 14), (6, 15, 13), (12, 13, 6). Ans: row minima 9, 6, 6 ⇒ (0 2 5), (0 9 7), (6 7 0); col min 0, 2, 0 ⇒ (0 0 5), (0 7 7), (6 5 0): 1→2, 2→1, 3→3 ⇒ cost = 11 + 6 + 6 = 23.
- 5) Differentiate the transportation and assignment problems. Ans: assignment has square matrix, supply = demand = 1, $x \in \{0, 1\}$.
- 6) What is an ε allocation? Ans: a tiny quantity placed to handle degeneracy.

> **Note:**
> Quick check: total of allocations in each row / column must equal supply / demand.

Quick Revision – Unit II

- Balanced: Σ supply = Σ demand; allocations = m + n – 1.
- IBFS: NWC (no cost), Least cost, VAM (penalty = two smallest difference).
- MODI: $u_i + v_j = c_{ij}$ for occupied; $\Delta = c - (u + v) \geq 0$ for all empty ⇒ optimal.
- Assignment: row reduce, column reduce, cover zeros, lines = n ⇒ assign.
- Dummy for unbalanced; max → subtract from largest.

Subject: OPTIMIZATION TECHNIQUES

Subject Code: 23AS2402e

UNIT III : Sequencing

Syllabus: Job sequencing, Johnson's algorithm for n jobs and two machines, n jobs and three machines, n jobs through m machines, two jobs through m machines.

I. Sequencing - Introduction

Definition:

Sequencing problem: n jobs have to be processed on m machines in a given technological order; we must find the order (sequence) of jobs that optimises a measure of performance - usually minimum ***total elapsed time***.

- Total elapsed time = time between the start of the first job on the first machine and the completion of the last job on the last machine.
- ***Idle time*** on a machine = time the machine is free between finishing one job and starting the next.
- Number of possible sequences for n jobs on m machines = $(n!)^m$. For 5 jobs on 2 machines: 120^2 - too many to enumerate.

Assumptions

- Each machine processes only one job at a time; each job on one machine at a time.
- Processing times are known and independent of the sequence.
- Once started a job is completed (no pre-emption); transfer time between machines is negligible.
- The order of machines for every job is the same (A then B then C ...) - for the n × m cases.

- Machines are ready and no breakdowns occur.

Notation

- t_{ij} = processing time of job i on machine j.
- Total elapsed time T = (finish time of last job on last machine).
- Idle time on last machine = T - Σ processing times on it (if the machine starts at 0).

> **Note:**
> Problem types in this unit: (a) n jobs, 2 machines; (b) n jobs, 3 machines; (c) n jobs, m machines; (d) 2 jobs, m machines.

2. Johnson's Algorithm - n Jobs, Two Machines

> **Definition:**
> Johnson's rule (S. M. Johnson, 1954) gives the optimal sequence minimising total elapsed time for n jobs through two machines M1 → M2 (every job first on M1, then on M2).

Algorithm

- Step 1: List the processing times of all jobs on M1 and M2.
- Step 2: Find the ***smallest*** time among all the times on both machines.
- Step 3: If it is on ***M1***, place that job at the ***first*** available position of the sequence; if it is on ***M2***, place it at the ***last*** available position.
- Step 4: Delete that job and repeat for the remaining jobs. Ties: break arbitrarily (ties between M1 and M2: put at M1 side, i.e. first).
- Step 5: Prepare the schedule, find total elapsed time and idle times.

> **Example:**
> Five jobs on two machines. Find the optimal sequence and elapsed time.

Job	1	2	3	4	5
M1	5	1	9	3	10
M2	2	6	7	8	4

- Smallest = 1 (Job 2 on M1) ⇒ first position: 2 _ _ _ _
- Next smallest = 2 (Job 1 on M2) ⇒ last position: 2 _ _ _ 1
- Next = 3 (Job 4 on M1) ⇒ second position: 2 4 _ _ 1
- Next = 4 (Job 5 on M2) ⇒ fourth position: 2 4 _ 5 1
- Remaining Job 3 fills the gap ⇒ **Optimal sequence: 2 - 4 - 3 - 5 - 1.**

Time table

Job	M1 in	M1 out	M2 in	M2 out
2	0	1	1	7
4	1	4	7	15
3	4	13	15	22
5	13	23	23	27
1	23	28	28	30

- Total elapsed time = **30** hours.
- Idle time M1 = 30 - 28 = **2**; Idle time M2 = 1 + 0 + 1 + 1 = **3** (check: 27 + 3 = 30 ✓).

> **Note:**
> A job on M2 starts at max(its finish on M1, finish of previous job on M2).

3. Another n Jobs, Two Machines Example

Example:

Six jobs; find the optimal sequence.

Job	A	B	C	D	E	F
M1	3	12	15	6	10	11
M2	8	10	10	6	12	1

- Smallest 1 (F on M2) ⇒ last: _ _ _ _ _ F
- Next 3 (A on M1) ⇒ first: A _ _ _ _ F
- Next 6 (D on M1 and D on M2 tie) ⇒ D on M1 first available: A D _ _ _ F
- Next 10 (B on M2, C on M2, E on M1): E on M1 ⇒ A D E _ _ F
- Remaining B (12, 10), C (15, 10): smallest 10 is on M2 for both: place B at last available (fifth), then C at fourth.
- **Sequence: A - D - E - C - B - F.**

Job	M1 in-out	M2 in-out
A	0 - 3	3 - 11
D	3 - 9	11 - 17
E	9 - 19	19 - 31
C	19 - 34	34 - 44
B	34 - 46	46 - 56
F	46 - 57	57 - 58

- Total elapsed time = **58**; idle M1 = 1; idle M2 = 3 + 2 + 3 + 2 + 1 = 11 (check: 47 + 11 = 58 ✓).

4. n Jobs and Three Machines

- Direct Johnson's rule works only for two machines. For three machines M1, M2, M3 we convert the problem into two fictitious machines G and H *if* one of

these conditions holds:

Formula:

Condition 1: $\min t_{i1} \geq \max t_{i2}$ OR Condition 2: $\min t_{i3} \geq \max t_{i2}$

- (the middle machine M2 is dominated by M1 or M3.)

Formula:

Fictitious machines: $G_i = t_{i1} + t_{i2}$, $H_i = t_{i2} + t_{i3}$

- Apply Johnson's rule to (G, H) to get the sequence; then compute the elapsed time using the ***original three machines***.
- If neither condition holds this method may not be optimal (use other methods).

Example:

Five jobs, three machines. Find the sequence and total elapsed time.

Job	1	2	3	4	5
M1	8	10	6	7	11
M2	5	6	2	3	4
M3	4	9	8	6	5

- $\min M1 = 6 \geq \max M2 = 6$ ✓ (Condition 1 satisfied).

Job	1	2	3	4	5
G = M1 + M2	13	16	8	10	15
H = M2 + M3	9	15	10	9	9

- Smallest = 8 (Job 3 on G) ⇒ first. Next 9 (H of jobs 1, 4, 5 – ties): place Job 1 last, Job 4 next from last, Job 5 next.
- Remaining Job 2 goes to position 2 ⇒ **Sequence: 3 – 2 – 5 – 4 – 1.**

Time table on the original machines

Job	M1 in-out	M2 in-out	M3 in-out
3	0 - 6	6 - 8	8 - 16
2	6 - 16	16 - 22	22 - 31
5	16 - 27	27 - 31	31 - 36
4	27 - 34	34 - 37	37 - 43
1	34 - 42	42 - 47	47 - 51

- ***Total elapsed time = 51.*** Idle: M1 = 51 - 42 = 9; M2 = 51 - 20 = 31; M3 = 51 - 32 = 19 (starting idle 8 included).

Note:

Lower bound check: Σ M1 + (M2 + M3 of last job) = 42 + 5 + 4 = 51 equals the answer, so the sequence is optimal.

5. n Jobs through m Machines

- Same idea as three machines: the order of machines is the same for all jobs, and the middle machines are "dominated".

Formula:

Conditions: $\min t_{i1} \geq \max t_{ij}$ (j = 2, ..., m-1) OR $\min t_{im} \geq \max t_{ij}$ (j = 2, ..., m-1)

- Convert to two fictitious machines:

Formula:

$G_i = t_{i1} + t_{i2} + \ldots + t_{i,m-1}$ (first m - 1 machines)
$H_i = t_{i2} + t_{i3} + \ldots + t_{im}$ (last m - 1 machines)

- Apply Johnson's rule on (G, H) to get the sequence; then compute the elapsed time on the original m machines.

Example:

Four jobs on four machines: A, B, C, D.

Job	1	2	3	4
A	13	8	9	10
B	5	3	4	6
C	4	5	3	2
D	12	9	10	11

- min A = 8, min D = 9; max of B = 6, max of C = 5. min A = 8 ≥ 6 ✓ (and 8 ≥ 5) - condition satisfied.

Job	1	2	3	4
G = A + B + C	22	16	16	18
H = B + C + D	21	17	17	19

- Smallest 16 (G of job 2 and job 3): put at first positions (job 2 first, job 3 second); next 17 (H of jobs 2, 3 - already placed); then Job 4 (18, 19): smallest is 18 on G ⇒ third; Job 1 (22, 21): last.
- **Sequence: 2 - 3 - 4 - 1.**

Job	A	B	C	D
2	0 - 8	8 - 11	11 - 16	16 - 25
3	8 - 17	17 - 21	21 - 24	25 - 35
4	17 - 27	27 - 33	33 - 35	35 - 46
1	27 - 40	40 - 45	45 - 49	49 - 61

- **Total elapsed time = 61.**

6. Two Jobs through m Machines (Graphical Method)

- Used for 2 jobs processed on m machines, each job with its own technological order (orders may differ).

Algorithm

- Step 1: Draw axes: horizontal axis = processing of Job 1 (machine times in order), vertical axis = processing of Job 2 (in its order).
- Step 2: For every machine draw the ***rectangle (block)*** formed by the intervals of Job 1 and Job 2 on that machine (both jobs cannot use it simultaneously).
- Step 3: Draw a path from the origin O (0, 0) to the end point (all of Job 1, all of Job 2) avoiding the interior of the blocks.
  - Move ***horizontally*** (only Job 1 works, Job 2 waits), ***vertically*** (only Job 2 works), or at ***45° diagonally*** (both work).
- Step 4: Choose the path with the ***maximum diagonal length*** (minimum horizontal and vertical movement).

Formula:

Total elapsed time = (total time of Job 1) + (sum of vertical segments) = (total time of Job 2) + (sum of horizontal segments)

Example:

Job 1: A (3), B (4), C (2). Job 2: B (2), A (5), C (3). Find minimum elapsed time.

Job 1 interval	A: 0 - 3	B: 3 - 7	C: 7 - 9
Job 2 interval	B: 0 - 2	A: 2 - 7	C: 7 - 10

- Blocks (x = Job 1, y = Job 2): A: (0 - 3) × (2 - 7); B: (3 - 7) × (0 - 2); C: (7 - 9) × (7 - 10).
- Best path: (0,0) → (2,2) diagonal → (3,2) horizontal 1 → (7,6) diagonal → (8,7) diagonal → (9,7) horizontal 1 → (9,10) vertical 3.

- Horizontal = 1 + 1 = 2; vertical = 3.
- Elapsed time = 9 + 3 = 12 = 10 + 2 ✓ ⇒ ***Minimum total elapsed time = 12 hours.***
- Interpretation: Job 2 waits 1 hour for machine A and 1 hour for machine C; Job 1 finishes at 9, Job 2 at 12.

> ***Note:***
> Blocks must never be crossed; diagonal moves along block edges are allowed.

7. Practice Problems with Answers

- 1) Two machines, 4 jobs: M1 = 4, 7, 3, 6; M2 = 5, 4, 8, 2. Find the sequence.
  - Ans: smallest 2 (job 4, M2) last; 3 (job 3, M1) first; 4 (job 1 on M1 and job 2 on M2): job 1 second, job 2 third ⇒ 3 – 1 – 2 – 4.
  - Times: job 3: M1 0–3, M2 3–11; job 1: M1 3–7, M2 11–16; job 2: M1 7–14, M2 16–20; job 4: M1 14–20, M2 20–22 ⇒ elapsed time 22.
- 2) For three machines with M1 = 3, 8, 7, M2 = 3, 4, 2, M3 = 5, 8, 6 – check the conditions. Ans: min M1 = 3 < max M2 = 4; min M3 = 5 ≥ 4 ✓ (condition 2) ⇒ G = 6, 12, 9; H = 8, 12, 8.
- 3) What is meant by idle time and total elapsed time?
- 4) State the assumptions of the sequencing problem.
- 5) How many sequences exist for 4 jobs on 3 machines? Ans: $(4!)^3$ = 13,824.
- 6) Why is Johnson's rule not directly applicable to 3 machines? Ans: M2 can become a bottleneck; conditions are needed to reduce to two machines.

> ***Note:***
> Always verify the elapsed time by a time table – do not trust only the sequence.

Quick Revision - Unit III

- Johnson: smallest time on M1 → first; on M2 → last; repeat.
- 3 machines: need min M1 ≥ max M2 or min M3 ≥ max M2; G = M1 + M2, H = M2 + M3.
- m machines: G = sum of first m - 1, H = sum of last m - 1.
- Idle time = elapsed time - total processing on that machine.
- Two jobs: graphical, elapsed time = total job time + vertical (or horizontal) movement.